COMP3821/9801

Workshop Week 10

Dynamic Programming

Copyright Disclaimer. *Reproducing, publishing, posting, distributing, translating this document is an infringement of copyright and will be referred to UNSW Student Conduct and Integrity for action.*

Guided Problem

The demonstrators will solve the following problems. For the rest of the workshop, work in small groups to solve the remaining workshop exercises. There are optional problems at the end of this document for further practice.

Guided Problem

Let $S[1..n]$ be a string of n characters. A *palindromic subsequence* is a subsequence of S that forms a palindrome. Recall that a palindrome that is read the same forwards and backwards, such as KAYAK. Describe and analyse a polynomial-time algorithm that returns the length of the longest palindromic subsequence of S.

For example, the length of the longest palindromic subsequence of DYNAMICPROGRAMMING is 7, as underlined.

Guided Problem

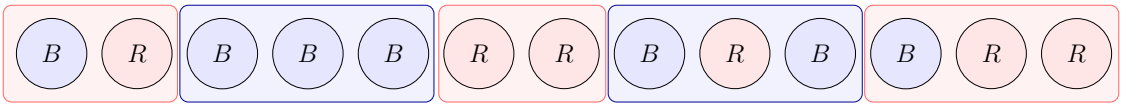
The city of UNSW has decided to partition High Street into voting districts. There are n houses on High Street, living on consecutive addresses labelled $1, 2, \ldots, n$. Each voting district is an interval of addresses $i, i+1, \ldots, j$ with $1 \leq i \leq j \leq n$. The city has enforced two constraints.

- Each house on High Street must belong to *exactly* one voting district.
- The number of houses in each district must lie between k and $2k$, where k is a parameter.

In each election, houses can vote for either OCEANIA or EURASIA but they can only vote for one party. Since the majority of houses in the city of UNSW support OCEANIA, the city deems a district *favourable* if more than half of the houses in the district support OCEANIA; otherwise, the district is *unfavourable*. Of course, the city has a complete record of all of the votes in the election.

The next election is coming up and the city has decided to ask you, as the budding algorithmist, to help assign districts. You are given a boolean array VOTE$[1..n]$, where VOTE$[i] = 1$ if resident i voted for OCEANIA and VOTE$[i] = 0$ if resident i voted for EURASIA. You are also given an integer k. Describe and analyse a polynomial-time (in terms of k and n) algorithm that returns the largest number of favourable districts in any legal partition of the houses.

For example, consider the following legal partition of $n = 13$ houses with $k = 2$ (therefore, each district can be assigned 2 to 4 houses). Houses who voted for OCEANIA are coloured blue and the houses who voted for EURASIA are coloured red. There are two favourable districts (coloured blue) and three unfavourable districts (coloured red).



The first district is unfavourable, the second district is favourable, the third district is unfavourable, the fourth district is favourable, and the fifth district is unfavourable.

Workshop Exercises

1. You are given an array $A[1..n]$ of n integers. Describe and analyse a polynomial-time algorithm that returns the length of the longest increasing subsequence of A whose elements sum to a multiple of 67.

   Note. *You may assume that we can compute modulo operations in constant-time.*

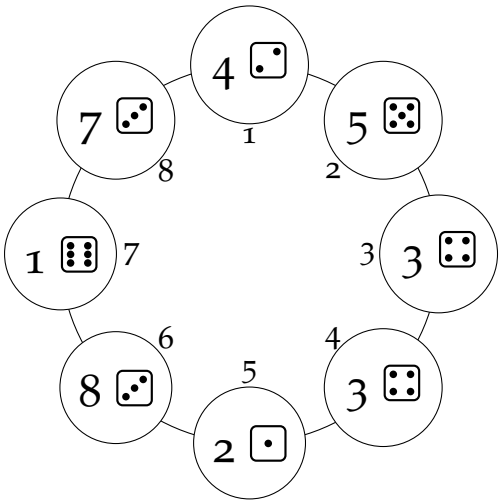
2. Let $S[1..n]$ be a string of n letters, representing a message in a foreign language that you do not understand! You have a blackbox translator ISWORD that can decide if a given string of letters w forms a coherent word in $O(|w|)$ time.

   You want to look for as many *non-overlapping* coherent words in S as possible. A *coherent garble* of S is a sequence of non-overlapping substrings of S such that each substring is a coherent word. The length of a coherent garble is the number of words it contains. Describe and analyse a polynomial-time algorithm that returns the length of the longest coherent garble of S.

3. Algopoly is a board game that takes place on a circular board with n spaces. Each space, numbered $i = 1, \ldots, n$ clockwise, has an associated positive score $s_i > 0$ and a positive dice value $d_i > 0$. The rules of the game are as follows.

   - The game begins at space 1 with a score of 0.
   - At each turn, roll a die.
     - If the die rolls a number that is *greater* than the die value d_i of the space you are currently on, then move one space clockwise.
     - Otherwise, move one space counterclockwise.
   - After rolling and moving, add the score value s_i of your new space to your current score.
   - Repeat the process until you eventually become bored.

   For example, consider the following board of size $n = 8$ and an iteration of an Algopoly game. Each state consists of a die value and a score value with the integer outside of the state representing its space number.



   - Begin at space 1 with a score of $S = 0$.
   - Roll a 2. Since $2 \leq d_1 = 2$, move to space 8 and earn $s_8 = 7$ points. Your current score is $S = 7$.
   - Roll a 5. Since $5 > d_8 = 3$, move to space 1 and earn $s_1 = 4$ points. Your current score is $S = 11$.
   - Roll a 3. Since $3 > d_1 = 2$, move to space 2 and earn $s_2 = 5$ points. Your current score is $S = 16$.
   - Get bored and end the game with a score of 16.

   A friend has given you all the s_i and d_i values of their Algopoly board and they have told you that they played a game with a final score of k. Describe and analyse a polynomial-time (in terms of n and k) algorithm that computes the *most likely* space that your friend finished the game on, or determine that it is impossible to finish the game on a score of k.

Optional Exercises

These exercises will not contain solutions. We encourage that you post on the forums to discuss the approaches for these problems.

4. A subset $S \subseteq \{1, \ldots, n\}$ is *independent* if it does not contain any pair of consecutive integers. Let A be an array of n positive integers. Describe and analyse a polynomial-time algorithm that computes the maximum sum of values that come from an independent subset of indices.

   Note. *The values of the array in an independent subset may be consecutive; spaced-ness is a property on the index set, not the actual values of the array.*

5. In this problem, we will design an algorithm that counts the number of ways to form a bit string of length n such that the 0 bits are always in groups of length k. To understand this problem statement, if $n = 3$ and $k = 2$, then possible strings are 111, 100, 001 since each string has the 0 bits occurring in groups of 2. Another example is: $n = 5$, $k = 2$, with the possible strings being 11111, 11100, 11001, 10011, 00111, 10000, 00100, 00001.

   Describe and analyse a polynomial-time algorithm that counts the number of strings such that the 0 bits are in groups of length k.

6. We consider the *Least Weight Subsequence* problem. Let $X[1..n]$ be an array of n integers. We are also given a function $\textsc{Weight}(i, j)$ whenever $i < j$. For any subsequence $S = \{i_1, \ldots, i_k\} \subseteq \{1, \ldots, n\}$, we define the *weight* of the subsequence to be

$$\textsc{Weight}(S) = \sum_{s=2}^{k} \textsc{Weight}(i_{s-1}, i_s).$$

   Describe and analyse an $O(n^2)$ algorithm that computes the cost of the least-weight subsequence.

   Note. *We can actually do better when the weight function is* ***concave****.*

7. You are travelling along a river by canoe. Along the way, you come across n trading posts and you can choose to rent a new canoe at any of these trading posts. The cost to rent a canoe from post i to post j is given by $F(i, j)$, where $1 \leq i < j \leq n$. You start canoeing from trading post 1 and you want to end at trading post n.

   Describe and analyse a polynomial-time algorithm to determine the smallest cost of travelling from trading post 1 to trading post n via a sequence of rented canoes.

8. Houses in the country of DynaProg are positioned on an $m \times n$ grid with the top left house positioned at $(1, 1)$ and the bottom right house positioned at (m, n). You live in the top left house and you need to get to your friend's house which is positioned in the bottom right, but you walk such that you're always making progress towards their house (i.e. you can only walk down or to the right).

   Describe and analyse an $O(mn)$ algorithm that counts the number of ways to get from your house to your friend's house, moving in only two directions.

9. You are emailed a text file containing important meeting notes. Unfortunately, the text file was corrupted and all of the white spaces of the text file has been removed. Therefore, all you see in the text file is a string of characters without

any spaces. Fortunately, your company is very good at keeping tabs with words that are mentioned during meetings and have devised a lookup function. The lookup function acts as an oracle, and returns "true" if and only if the string is a valid word; that is, given a word w, LOOKUP(w) returns TRUE if and only if w is a valid word.

Given a string s of n characters, describe and analyse a polynomial-time algorithm that determines whether s forms a valid text by placing spaces in between characters to form a string of valid words.

10. Let $G = (V, E)$ be a directed and acyclic graph. A subset $S \subseteq V$ of vertices is called *non-connected* if any pair of distinct vertices $u, v \in S$ are not connected by a directed path. Describe and analyse an $O(m + n)$ algorithm to compute the smallest number of non-connected sets $S_1, \ldots, S_k$ such that $S_1 \cup \ldots \cup S_k = V$.

    Hint. *Define a suitable partial order on the set of vertices; what is a suitable definition of a chain and an antichain under this partial order?*

11. Given a positive integer n, the *complexity* of n is the minimum number of ones that can be used to represent n, using only the operations of addition and multiplication, as well as parenthesisation.

    For example, we have the following representations:

$$
\begin{aligned}
6 &= (1+1+1) \times (1+1). \\
8 &= (1+1) \times (1+1) \times (1+1). \\
9 &= (1+1+1) \times (1+1+1). \\
12 &= (1+1+1+1) \times (1+1+1). \\
19 &= (1+1+1) \times (1+1+1) \times (1+1) + 1.
\end{aligned}
$$

    The first twenty entries are given for you.

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
	1	2	3	4	5	5	6	6	6	7	8	7	8	8	8	8	9	8	9	9

    (a) Show that every positive integer can be represented by a string of ones, along with addition, multiplication, and parenthesisation operations; that is, the complexity of n is always finite.

    (b) Given a positive integer n, describe and analyse an $O(n^2)$ algorithm to compute the minimum number of one's (1's) using only the operations of addition and multiplication, as well as parentheses, whose expression equals n.

    Note. *This is also known as the Mahler-Popken complexity. Here is the OEIS entry.*

12. As a lover of puzzles, you have come up with a new strategy puzzle game called INTERVALBLAST. In this game, you are given a row of n squares, where each square is labelled with a positive integer. On each turn, you need to choose two squares with no missing square in between them. Once you choose the two squares, you earn points equal to the product of the integers on the labels. Afterwards, your two chosen squares *and* all of the squares in between are removed. Once a square is removed, it cannot be chosen in a future round. The game ends when there are no more valid turns left or if there are no more squares left to

choose, and the aim of the game is to maximise the number of points by the end of the game.

To see this in action, consider the following game; each row shows a distinct turn. The red underlined numbers are the two squares chosen on that turn, and the grey squares are the ones which were removed in earlier turns. A partial score **prior** to the turn is supplied. No moves are possible on Turn 4.

Turn	Interval								Partial Score
1	3	1	4	1	5	9	2	6	0
2	3	1	4	1			2	6	45
3				1			2	6	57
4				1					69

In this game and following sequence of moves, the number of points earned is

$$5 \times 9 + 3 \times 4 + 2 \times 6 = 45 + 12 + 12 = 69.$$

Note that this is not necessarily the highest possible number of points that could be earned for this specific instance.

Describe and analyse a polynomial-time algorithm that computes the maximum number of points that can be earned in a given instance of IntervalBlast.

13. You are given an array A of n integers. Some of these elements are marked as *dodgy*, and you want to find the length of the longest increasing subsequence that include at most k dodgy elements. Given an array $A[1..n]$, a boolean array Dodgy$[1..n]$, and an integer k, describe and analyse an $O(kn^2)$ algorithm to compute the length of the longest increasing subsequence that contains at most k dodgy elements.

    For example, if we have the array

    $$A = [3^*, 1, 4^*, 1^*, 5, 9, 2^*, 6],$$

    where all of the dodgy elements are marked by an asterisk $(*)$, then a longest increasing subsequence containing at most 3 dodgy elements is $\{3^*, 4^*, 5, 9\}$. Your algorithm should, then, return 4.

14. A *partition* of an integer n is a sequence $\langle p_1, \ldots, p_t \rangle$ such that $1 \leq p_1 \leq p_2 \leq \cdots \leq p_t \leq n$ and $p_1 + \cdots + p_t = n$. We call the p_k terms *parts*.

    (a) Describe and analyse a polynomial-time (in terms of k and n) algorithm that computes the number of partitions of n in which every part is at most k, where $1 \leq k \leq n$.

    (b) Hence, explain how to compute the total number of partitions of n.

15. Skiers go fastest with skis whose length is about their height. Your team consists of n members, with heights $h_1, \ldots, h_n$. Your team gets a delivery of $m \geq n$ pairs of skis, with lengths $\ell_1, \ldots, \ell_m$. Your goal is to assign each skier a pair of skis such that the sum of the absolute differences between the height h_i of the skier and the length of the corresponding ski they obtained is minimised. In other words, suppose that $s(i)$ denotes the index of the ski that is assigned to skier i. Then we want to minimise

    $$\sum_{i=1}^{n} |h_i - \ell_{s(i)}|.$$

Describe and analyse an $O(mn)$ algorithm that computes the minimum cost.

16. You are given two arrays $A[1..n]$ and $B[1..m]$. For subsequences $A'[1..k]$ and $B'[1..k]$ of the *same* length, the *dot product* of A' and B' is equal to the sum of the pairwise products of A' and B'. In other words, the dot product of A' and B' is given by
$$A' \cdot B' = A'[1] \times B'[1] + \cdots + A'[k] \times B'[k].$$

    Given two arrays $A[1..n]$ and $B[1..m]$, describe and analyse a polynomial-time algorithm to compute the maximum dot product between two *non-empty* subsequences of A and B.

    Note. *The subsequences have to be of the same length; otherwise, the dot product is not well-defined.*

17. A *shuffle* of two strings X and Y is formed by interspersing the characters into a new string, keeping the characters of X and Y in the same order.

    (a) Given three strings $X[1..m]$, $Y[1..n]$, and $Z[1..m+n]$, describe and analyse a polynomial-time algorithm to decide if Z is a shuffle of X and Y.

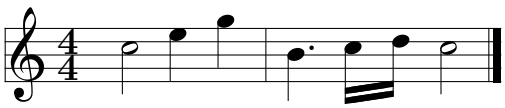
    (b) Given three strings $X[1..m]$, $Y[1..n]$, and $Z[1..m+n]$, describe and analyse a polynomial-time algorithm to count the number of ways that X and Y can be shuffled to form Z.

18. You are a pianist preparing for an upcoming competition. You do not have a lot of time to prepare your repertoire. As a fan of algorithms, you want to devise a strategy to optimally play your repertoire using the *least painful* fingering.

    We make the following simplifying assumption:

    > Certain notes are to be played longer than others. In our problem, we ignore this requirement and only focus on hitting the *correct* notes.

    Formally, you are given a sequence $A = [a_1, \ldots, a_n]$ of n notes and a *difficulty* oracle $d(f, a, g, b)$, which measures how difficult it is to go from note a with finger f to note b with finger g.



The first two bars to Mozart's Sonata No. 16 in C Major, K. 545.

    For example, the extract above can be expressed with the array $A = [C, E, G, B, C, D, C]$. Describe an $O(n)$ algorithm to return the value of the *least* difficult way to play the piece described by the array A.

    Note. *For this problem, we have five fingers.*

19. In the arcade game *Dance Dance Revolution* (DDR), players stand on a stage and hit arrows as they scroll across the screen. More specifically, a sequence of n arrows (⬅, ⬆, ➡, ⬇) will scroll across the screen, and as each arrow hits the top of the screen, the player must stand on the corresponding arrow on the stage.

    We play a variant of DDR, aptly named *Don't Dance Revolution* (DDR2), where the goal is to play the game like DDR but move as little as possible. The game

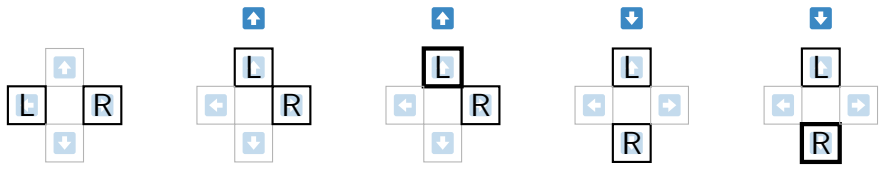
plays like DDR except when an arrow reaches the top of the screen and the player already has a foot on the correct arrow, then the player is awarded one lazy point. If neither foot is on the correct arrow, then the player must move *exactly* one foot from its current location to the correct arrow on the platform.

Unfortunately, the game is a bit unforgiving: any wrong move will cause the player to lose the game and *all* of their lazy points. Wrong moves include:

- Failing to step on the correct arrow.
- Moving more than one foot at any given time.
- Moving either foot when the player is already stepping on the correct arrow.

You are given a sequence A of n arrows. Assume that your left foot starts on ⬅ and your right foot starts on ➡, and you have memorised the entire sequence of arrows.

For example, consider the following sequence: ⬆ ⬆ ⬇ ⬇. We can earn up to two lazy points as follows:



Start game! *Lazy point!* *Lazy point!*

(a) Show that it is always possible to earn at least $\lfloor n/4 \rfloor$ lazy points during a round of DDR2.

(b) Describe and analyse an $O(n)$ algorithm to determine the maximum number of lazy points you can earn in a round of DDR2.

Hint. *We only have two feet...*

20. Let $S[1..n]$ be a string of n characters. A *palindromic subsequence* is a subsequence of S that also forms a palindrome. Recall that a palindrome is a string that is read the same forwards and backwards, such as KAYAK and RACECAR.

A *twin palindrome* is the concatenation of two *non-empty* palindromes, such as KAYAKKAYAK and AMANAPLANACANALPANAMARACECAR. Describe and analyse a polynomial-time algorithm that returns the length of the longest subsequence of S that forms a ***twin palindrome***.

For example, consider the string MAYBEDYNAMICPROGRAMMING. The length of the longest twin palindromic subsequence is 12, given by the subsequence

MAYBEDYNAM ICPROGRAMMING.

Hint. *You do not need dynamic programming for this.*

21. You are given two strings $S = s_1 \dots s_m$ and $T = t_1 \dots t_n$. We define a cost function $C(i, j)$ on pairs of letters, one coming from S and another coming from T. Your goal is to merge S and T together to form a string A while maintaining the ordering of the letters from S and T; that is, if $S =$ abcd and $T =$ efgh, then one possible string for A is $A =$ abefcghd while $A' =$ abefchgd is not possible because h comes after g in T.

We define the *cost* of a merge of S and T to be the sum of the costs $C(s_i, t_j)$ of adjacent letters which come from different strings. For example, if A contains s_k and s_{k+1} as adjacent letters, then there is no cost. However, the cost of $s_k t_j$ is given by $C(s_k, t_j)$. You may also assume that the cost function is symmetric; that is, $C(s_k, t_j) = C(t_j, s_k)$.

Describe and analyse an $O(mn)$ algorithm to compute the minimum cost to merge S and T into one string.

22. (a) A *palindromic decomposition* of a string w is a partition of w into substrings $u_1, \ldots, u_m$ such that $w = u_1 \ldots u_m$ and each part u_i forms a palindrome. For example, if $w = \texttt{abcbabaab}$, then $u_1 = \texttt{a}, u_2 = \texttt{bcb}, u_3 = \texttt{a}, u_4 = \texttt{baab}$ is a palindromic decomposition of w.

    We assign each substring u_i a cost denoted by $\mathsf{cost}(|u_i|)$, where $|u_i|$ is the length of u_i. Finally, we define the cost of a decomposition $u_1, \ldots, u_m$ to be the sum of the costs of the individual substrings u_i; that is, if $w = u_1 \ldots u_m$, then the cost of the decomposition is

    $$\mathsf{cost}(|u_1|) + \cdots + \mathsf{cost}(|u_m|).$$

    We want to find the minimum cost of a *palindromic decomposition* of w. Given a string $w = w_1 \ldots w_n$ and an array of all costs, describe and analyse an $O(n^2)$ algorithm to compute the minimum cost of a palindromic decomposition of w.

    (b) A *palindromic palindromic decomposition* is a palindromic decomposition such that the sequence of length is itself a palindrome. For example, consider the string

    BOBSMAMASEESAUKULELE.

    We can decompose this string into:

    BOB S MAM ASEESA UKU L ELE,

    where the sequence of lengths is $(3, 1, 3, 6, 3, 1, 3)$ which is a palindrome. Any string always has a decomposition via the decomposition $(1, 1, \ldots, 1)$. Describe and analyse an $O(n^2)$ algorithm to compute the minimum number of palindromes in any palindromic palindromic decomposition.

23. Let S be an array of n distinct integers. Describe and analyse an $O(n^2)$ algorithm to compute the smallest number of *decreasing* subsequences that partition S.

    Hint. *You can do this directly with dynamic programming. There is, however, an easier way... it might help to solve the previous problem first.*

24. There are n restaurants lined up in a row. The ith restaurant has size $S[i]$. The goal is to merge these restaurants together such that the final restaurant is a single large restaurant. However, you can only merge restaurants in the following manner:

    - You can only merge two adjacent restaurants at a time, so that the new restaurant is the physical union of two buildings.
    - The size of the new restaurant is the sum of the two restaurants being merged.
    - The cost of a merge is equal to the size of the larger restaurant.

The *total cost* of a sequence of merges is the sum of the costs of the individual merges.

(a) Let S be an array of n elements. Implement the function $\textsc{Range}(i, j)$ that computes the value

$$\textsc{Range}(i, j) = S[i] + \cdots + S[j].$$

Your implementation should take $O(n)$ time, so that $\textsc{Range}(i, j)$ can be called in $O(1)$ time.

(b) Hence, describe and analyse a polynomial-time algorithm that computes the minimum total cost of creating one large restaurant from the original n restaurants.

25. Tailor Thrift is a brand new clothing store opening up in your local town. It has assigned you as its inaugural general manager, and have asked you to come up with a dynamic pricing strategy to compete with currently established clothing stores.

Your pricing strategy is as follows:

- All clothes in the store have the same advertised price at all times. You can set the advertised price at the start of the day and you can *decrease* the price later on.
- Customers come into your store one at a time.
  - If the customer is willing to pay at the same price as your advertised price, then they will buy one of your clothes at your advertised price.
  - If your advertised price is too high, the customer will suggest a lower price that they are willing to pay. If you refuse to lower your advertised price, then the customer will walk out of the store without buying anything.
- When you lower your advertised price, the new price will stay in effect until the price has been lowered again or until the end of the day. *You can never increase your advertised price*.

With your close knowledge of the people in the town, you can accurately predict when customers will come into the store and how much each customer is willing to pay for clothes.

Formally, you are given an array $\textsc{Value}[1..n]$, where $\textsc{Value}[i]$ denotes the amount that customer i is willing to pay for a clothing item in Tailor Thrift. Describe and analyse a polynomial-time algorithm that computes the maximum amount of money Tailor Thrift can earn by selling clothing items using the dynamic pricing strategy.

For example, if the input array is $\textsc{Value} = [5, 3, 1, 4, 2]$, then your algorithm should return 13 because you can earn $5 + 3 + 0 + 3 + 2 = 13$ dollars using the advertised prices $[5, 3, 3, 3, 2]$ and this is the optimal strategy.

26. Let S be a sequence of integers. A *tribus* in S is a triplet of consecutive sequence elements whose sum is a multiple of 3. For example, if

$$S = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 7],$$

then all of the tribuses are highlighted below:

$$[3, \underline{1, 4, 1}, 5, 9, 2, 6, 5, 3, 7]$$
$$[3, 1, 4, \underline{1, 5, 9}, 2, 6, 5, 3, 7]$$
$$[3, 1, 4, 1, 5, 9, 2, 6, \underline{5, 3, 7}]$$

A sequence A is said to be *more tribussy* than a sequence B if A has more tribuses than B. Given a sequence of n integers S, describe and analyse a polynomial-time algorithm to compute the number of tribuses in the *most tribussy* subsequence of S.

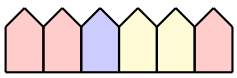
For example, the sequence $S = [0, 1, 1, 2, 3, 5, 8, 13, 21]$ should return four, because its most tribussy subsequence $S' = [0, 1, 2, 3, 8, 13, 21]$ has the following four tribuses.

$$\left[\underline{0, 1, 2}, 3, 8, 13, 21\right]$$
$$\left[0, \underline{1, 2, 3}, 8, 13, 21\right]$$
$$\left[0, 1, 2, \underline{3, 8, 13}, 21\right]$$
$$\left[0, 1, 2, 3, \underline{8, 13, 21}\right]$$

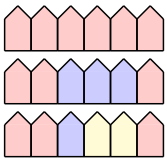
27. Gerald has a fence that he wants to get painted, which is composed of n wooden slats, all initially unpainted. Each wooden slat has been designated a specific colour.

    However, Gerald does not want to do the painting himself. On each day, he can hire one worker to paint any *contiguous* subset of slats a single colour. Since the paint is of awesome quality, Gerald does not care how many times a wooden slat gets painted. The only thing that he cares about is whether the last paint colour matches the designated colour of the wooden slat.

    For example, consider the following instance.



    We have the following painting plan.



    This painting plan requires three days which is optimal. Given an array of designated colours $C[1, \ldots, n]$, describe and analyse an $O(n^4)$ algorithm to determine the minimum number of days Gerald must spend to have his fence painted.

    Note. *Gerald can only hire one worker on each day.*

28. You are managing a ferry that is sailing across the Yarra River. The ferry has three lanes and each lane is L metres long. As the ferryloader, your job is to load the vehicles onto the ferry by directing it to one of the three lanes. The vehicle must then park as far forward as possible in that lane and vehicles enter the ferry in the order they arrive. Furthermore, if the vehicle currently at the front

of the queue cannot board the ferry because it cannot fit into any of the three lanes, then no more vehicles can board the ferry.

Since it is your boss' ferry company and for *some* reason you dislike your boss, you want to load as *few* vehicles onto the ferry as possible for each trip. However, you don't want to be too suspicious about your malpractice. Therefore, if the vehicle that is currently at the front of the queue can fit into some lane, it *must* be assigned to a lane where it fits. Finally, you can see the lengths of all of the vehicles in advance through your security camera.

Formally, you are given an array $X[1..n]$, where $X[i]$ denotes the length of vehicle i with $1 \leq X[i] \leq L$. You may assume that $X[i]$ is a positive integer. Describe and analyse an $O(L^3)$ algorithm that computes the smallest integer k such that you can fit vehicles 1 to k on the ferry in such a way that vehicle $k+1$ does not fit.

29. Let $G = (V, E)$ be a directed and weighted graph, where each edge $e \in E$ has positive weight $w(e) > 0$. Each edge is coloured either red or blue. We say that a cycle is *scrumptious* if less than a third of its edges are red. In other words, if a cycle contains r red edges and b blue edges, then it is scrumptious if and only if $2r < b$. Describe and analyse an $O(|V| \cdot |E|)$ algorithm to decide if G contains a scrumptious cycle.

30. An *arbitrage* is a money-making scheme that utilises the difference in currency exchanges to make a profit. For example, consider the following currency exchanges:

    - 1 Australian dollar is equal to 120 Chinese yuan.
    - 1 Chinese yuan is equal to 0.01 US dollars.
    - 1 US dollar is equal to 1.2 Australian dollars.

    Then a person from the Australian market can exchange currencies and make a profit by exchanging 1 Australian dollar to Chinese yuan, followed by Chinese yuan to US dollars, and finally, from US dollars to Australian dollars, ending with 1.44 Australian dollars, which is a profit! The cycle of currencies being exchanged is called an *arbitrage cycle*. We would like to design an algorithm to detect if there is an arbitrage cycle currently in the market.

    Formally, you know that n currencies are currently being traded and you are given a two-dimensional array R containing exchange rates between every pair of currencies; that is, for currencies i and j, one unit of currency i can be traded for $R[i, j]$ units of currency j. You **cannot** assume that $R[i, j] \cdot R[j, i] = 1$.

    (a) Suppose that $i \to j \to k \to \ell \to i$ is an arbitrage cycle; that is, we have that

    $$R[i, j] \cdot R[j, k] \cdot R[k, \ell] \cdot R[\ell, i] > 1.$$

    Modify the array R to obtain a new array L so that if

    $$R[i, j] \cdot R[j, k] \cdot R[k, \ell] \cdot R[\ell, i] > 1,$$

    then

    $$L[i, j] + L[j, k] + L[k, \ell] + L[\ell, i] < 0.$$

    (b) Hence or otherwise, describe and analyse a polynomial-time algorithm to detect an arbitrage cycle.

    Hint. *The Bellman-Ford algorithm does not just compute shortest paths...*

31. Let $G = (V, E)$ be a directed and weighted graph on n vertices. In other words, $|V| = n$. For two vertices $u, v \in V$, we define the *distance* $\delta(u, v)$ from u to v to be the weight of the shortest path from u to v. However, we alter the distances from u to v for two special cases:

    - If there is no path from u to v, define $\delta(u, v) = \infty$.
    - If there is a path from u to v that contains a negative-weight cycle, define $\delta(u, v) = -\infty$.

    Given a graph G, describe and analyse a polynomial-time algorithm that computes $\delta(u, v)$ for all pairs $u, v \in V$.

    Hint. *Note that Floyd-Warshall's algorithm assumes that G contains no negative-weight cycles. How would you modify the algorithm?*

32. Let $G = (V, E)$ be an directed and weighted graph, and let $\Gamma = (e_1, \ldots, e_k)$ be a directed cycle in G. We define the *weight* of Γ as the sum of the weights of the edges in Γ; that is, we have that

$$w(\Gamma) = \sum_{i=1}^{k} w(e_i),$$

    where $w(e_i)$ denotes the weight of edge e_i in G.

    (a) Prove that computing the minimum weighted cycle is NP-hard.

    (b) We now turn to a different metric called the *minimum mean weighted cycle*. Instead of computing the minimum weighted cycle, we look to approximate it by computing the weight of the cycle Γ that minimises the quantity

$$\hat{w}(\Gamma) = \frac{w(\Gamma)}{|\Gamma|},$$

    where $|\Gamma|$ denotes the length of the cycle.

    Describe and analyse a polynomial-time algorithm that computes the minimum mean weighted cycle.

    Note. *The minimum mean weighted cycle is used to compute minimum-cost flows; specifically, it is used to in the minimum mean cycle cancelling algorithm.*

33. Let $G = (V, E)$ be a directed and weighted graph, where $w(e)$ denotes the weight of edge e. Note that $w(e)$ can be positive, negative, or zero. Additionally, each edge is coloured one of three colours: red, blue, or green. These colours denote the permitted operations that we can do on the weights of the edges as follows.

    - A red edge denotes an edge where we can increase but not decrease the weight of the edge.
    - A blue edge denotes an edge where we can decrease but not increase the weight of the edge.
    - A green edge denotes an edge where we cannot modify the weight of the edge.

    We now consider the following problem.

    CycleNullification

Input. A directed and weighted graph $G = (V, E)$, where every edge is coloured either red, blue, or green.

Task. Is it possible to modify the edge weights of G such that every cycle of G has weight zero, subject to the edge colour constraints?

For brevity, we will say that G is *cycle nullifiable* if the edge weights of G can be modified so that every cycle of G has weight zero subject to the edge colour constraints.

(a) Let $G' = (V, E')$ be the modified graph, where

- we construct the reversal edge $v \to u$ for every red or green edge $u \to v$ with weight $w(v \to u) = -w(u \to v)$, and
- we remove all of the red edges from the graph.

Prove that G is cycle nullifiable if and only if there are no negative cycles in G'.

(b) Hence, describe and analyse a polynomial-time algorithm to decide if G is cycle nullifiable.

34. We are given a directed and weighted graph $G = (V, E, w)$, where $w(e) > 0$ for each $e \in E$. Additionally, each edge is coloured either red or blue.

(a) Let $\textsc{onlyRedEdges}(u, v, \ell)$ denote the weight of the shortest path from u to v that uses *only* red edges and have length at most ℓ. For a fixed pair of points u and v, describe and analyse an $O(n^3 \log k)$ algorithm to compute $\textsc{onlyRedEdges}(u, v, k-1)$.

Hint. *You should only need $O(n^2 \log k)$ subproblems.*

Hint 2. *Alternatively... you can just apply Bellman-Ford.*

(b) Hence, describe and analyse $O(n^3 \log k)$ algorithm that returns a *closed walk* with the smallest weight that contains *at least* one blue edge and does not contain k consecutive red edges.

A *closed walk* is a walk that starts and ends at the same vertex.

35. Let $T = (V, E)$ be an undirected tree (i.e. an undirected, connected, and acyclic graph). Describe and analyse an $O(|V|)$ algorithm that computes the smallest number of *vertex*-disjoint paths that cover all of the vertices of T.

Note. *Can you find a simpler greedy algorithm?*

36. Let A be an $m \times n$ array of real numbers. We say that A satisfies the *Monge property* if, for all i, j, k, ℓ such that $1 \le i < k \le m$ and $1 \le j < \ell \le n$, we have that

$$A[i, j] + A[k, \ell] \le A[i, \ell] + A[k, j].$$

An array that satisfies the Monge property is called a *Monge array*. In other words, whenever we pick two rows and two columns of a Monge array and consider the four elements a the intersections of the row and columns, the sum of the upper-left and lower-right elements is at most the sum of the lower-left and upper-right elements.

(a) Prove that A satisfies the Monge property if and only if, for all $i = 1, \ldots, m-1$ and $j = 1, \ldots, n-1$, we have that

$$A[i, j] + A[i+1, j+1] \le A[i, j+1] + A[i+1, j].$$

(b) Let A be an $m \times n$ array. For each row $1 \leq i \leq m$, let $f(i)$ denote the index of the column containing the leftmost minimum element of row i. For any Monge array A, prove that

$$f(1) \leq f(2) \leq \cdots \leq f(m).$$

(c) Hence, describe and analyse a divide-and-conquer algorithm that computes the leftmost minimum element in each row of an $m \times n$ Monge array A in $O(m + n \log m)$ time.

Hint. *Construct a submatrix A' of A that consists of the even-numbered rows of A and recursively compute the leftmost minimum for each row in A'. Explain how to compute the leftmost minimum element of each odd-numbered row of A in $O(m + n)$ time.*

Note. *This is known as the SMAWK algorithm.*

37. Let $X[1..n]$ be a sorted array of distinct real numbers and let k be an integer. For any pair of indices i, j such that $i < j$, let $\textsc{area}(i, j) = (j-i)\cdot(X[j]-X[i])$ denote the area of a histogram rectangle representing the $j - i$ items in the interval $X[i..(j-1)]$. We can define a *histogram* for X determined entirely by a sorted array $B[0..k]$ of distinct breakpoint indices such that $B[0] = 1$ and $B[k] = n$. Our goal is to choose these breakpoints such that the sum of the squared areas of its rectangles is minimised; that is, we want to compute an array B such that

$$\textsc{Cost}(B) = \sum_{i=1}^{k} (\textsc{area}(B[i-1], B[i]))^2$$

is minimised.

(a) Let $A[1..n, 1..n]$ be an upper-triangular matrix such that

$$A[i, j] = (\textsc{area}(i, j))^2$$

if $i < j$ and $A[i, j]$ is undefined everywhere else. Show that A is a Monge array.

Hint. *It suffices to check the Monge property where $k = i + 1, \ell = j + 1$.*

(b) Describe and analyse an $O(n \log n)$ algorithm that finds the minimum element in every row of an $n \times n$ upper triangular Monge array.

(c) Describe and analyse an $O(kn^2)$ algorithm that computes a minimum cost histogram with k number of rectangles. Reduce the complexity to $O(kn \log n)$.